LAPORAN PRAKTIKUM BIG DATA

Pengenalan Big Data & Lingkungan Kerja, serta Pengolahan Dataset Skala Besar Menggunakan Python

Disusun Oleh:

Stanley Leonard Darmali/Kelas B

2311532021

Dosen Pengampu:

Afdhal Dinilhak

Ajeng Dwi Asti

Departemen Informatika

Fakultas Teknologi Informasi

Universitas Andalas

2026

Repository Github:
https://github.com/AbyssWatcher07/BigData26_B_2311532021_StanleyLeonardDarmali

1. Langkah-langkah Kerja (J-1 s.d. J-10)

J-1. Verifikasi Lingkungan Kerja: Memeriksa dan mendokumentasikan spesifikasi mesin virtual Google Colab, melingkupi versi Python (3.13.18), OS (Linux 6.6.159), jumlah core CPU (2 core), ketersediaan RAM fisik (12 GiB Total / 10 GiB Available), kapasitas disk, versi pustaka utama (pandas 2.2.3, numpy 2.4.6, pyarrow 23.0.1, matplotlib 3.10.0), serta Java OpenJDK (21.0.12.1).

J-2. Menghubungkan Google Drive & Struktur Folder: Mengintegrasikan Google Drive ke direktori runtime Colab (/content/drive) serta membuat folder kerja lokal (/content/data) dan folder penyimpanan permanen di Drive (/content/drive/MyDrive/BigData/Praktikum1) untuk memastikan semua artefak tersimpan secara permanen.

J-3. Menyiapkan Alat Ukur Waktu dan Memori: Membangun fungsi utilitas ukur() menggunakan pustaka psutil dan time untuk mencatat durasi eksekusi (detik) serta pertumbuhan pemakaian RAM fisik (Resident Set Size / RSS dalam MB) secara kuantitatif.

J-4. Mengunduh Dataset & Inspeksi Metadata: Mengunduh dataset biner Parquet NYC Yellow Taxi bulan Januari 2023 (yellow_tripdata_2023-01.parquet, ukuran 45.5 MB) secara terprogram, kemudian membaca metadatanya menggunakan pyarrow.parquet tanpa memuat baris data ke RAM (tercatat 3.066.766 baris, 19 kolom, 1 row group).

J-5. Memuat Data & Optimasi Memori: Membandingkan pemakaian memori antara memuat seluruh 19 kolom (565.6 MB RAM) berbanding 8 kolom spesifik (187.2 MB RAM). Selanjutnya, tipe data dioptimalkan melalui downcasting numerik ke float32 dan pengubahan payment_type ke category/int8, yang memangkas pemakaian RAM menjadi 119.9 MB (penghematan 35.9%).

J-6. Pembersihan Data (Veracity): Menghitung metrik durasi_menit dan menyaring baris anomali (jarak <= 0, durasi <= 0 atau > 180 menit, serta total_amount <= 0). Penyaringan berhasil membersihkan 79.855 baris anomali (2.60%), meninggalkan 2.986.911 baris data layak analisis.

J-7. Pemrosesan Out-of-Core (Chunking): Menyimulasikan kondisi file besar dengan mengekspor data bersih ke CSV (260 MB). Pemrosesan agregasi per jam dilakukan secara bertahap menggunakan pd.read_csv(..., chunksize=500000), menjaga pertumbuhan memori tetap rendah (+19.0 MB RSS delta).

J-8. Pemrosesan Terdistribusi (PySpark Local Mode): Menjalankan Apache Spark 3.5.1 mode lokal (local[*]), membuat DataFrame terdistribusi, mengaplikasikan penyesuaian tipe

tanggal TIMESTAMP_NTZ via timestamp casting, serta mengeksekusi agregasi per jam dan kueri SQL per metode pembayaran untuk mengevaluasi lazy evaluation dan rencana eksekusi (explain plan).

J-9. Agregasi Akhir & Penyimpanan Artefak: Menghitung tabel agregasi final per jam (jumlah perjalanan, rata-rata jarak, rata-rata durasi, rata-rata tarif, dan median tarif) menggunakan pandas. Artefak disalin dan disimpan permanen ke Drive dalam format agregasi_per_jam.csv, agregasi_per_jam.parquet, dan pengukuran_kinerja.csv.

J-10. Visualisasi Hasil: Membuat diagram batang (bar chart) distribusi volume perjalanan per jam menggunakan Matplotlib dan mengekspor hasilnya sebagai file gambar distribusi_per_jam.png ke Google Drive.

2. Analisis Hasil (N.1 s.d. N.6)

N.1 Biaya Membaca Data

Membaca seluruh 19 kolom memakan memori sebesar 565.6 MB, sedangkan membaca 8 kolom terpilih hanya memakan 187.2 MB, menghasilkan rasio penghematan memori 3.02x. Pengurangan kolom (19 ke 8) memiliki rasio 2.38x, dan durasi eksekusi berkurang dari 1.30 detik menjadi 1.00 detik (rasio 1.30x). Rasio memori (3.02x) melebihi rasio kolom (2.38x) karena 11 kolom yang dibuang mencakup kolom bertipe data lebar (8-byte float64 dan timestamp) serta overhead metadata. Penghematan I/O disk terjadi secara signifikan karena Parquet menerapkan column pruning, sehingga blok data kolom yang tidak dipanggil tidak dibaca dari disk.

N.2 Efek Optimasi Tipe Data

Pengubahan tipe data berhasil menghemat RAM dari 187.2 MB menjadi 119.9 MB (penghematan 35.9%). Kontribusi terbesar berasal dari downcasting 4 kolom float64 (trip_distance, fare_amount, tip_amount, total_amount) ke float32 yang memangkas alokasi 4 byte per nilai (~12.26 MB per kolom pada 3,06M baris). Pengubahan payment_type ke category (int8) juga menghemat kolom kategorikal. Risikonya adalah penurunan presisi desimal desimal (single precision, ~7 digit signifikan), yang aman untuk analisis agregat deskriptif tetapi tidak boleh digunakan untuk pencatatan transaksi keuangan/billing.

N.3 Format Penyimpanan (Parquet vs CSV)

File Parquet terkompresi berukuran 45.5 MB, sedangkan file CSV uncompressed membengkak hingga ~260 MB (CSV 5.7x lebih besar). Penyebab utama: (1) Parquet menyimpan data per kolom dan menerapkan algoritma kompresi Snappy/Gzip serta dictionary encoding pada nilai berulang; (2) CSV menyimpan setiap nilai sebagai karakter teks ASCII (membutuhkan byte terpisah untuk setiap digit/titik koma), sedangkan Parquet menyimpan angka dalam bentuk biner terkompresi yang jauh lebih efisien.

N.4 Pemrosesan Out-of-Core (Chunking)

Memuat data secara utuh menyebabkan pertumbuhan memori (delta RSS) sebesar +221.6 MB, sedangkan agregasi chunking (500.000 baris/potongan) hanya meningkatkan delta RSS sebesar +19.0 MB. Trade-off yang diterima adalah durasi pemrosesan yang lebih lama (4.88 detik via CSV chunking vs 1.00 detik via Parquet in-memory) serta keterbatasan dalam menghitung metrik non-asosiatif (seperti median eksak) secara langsung per potongan.

N.5 Komparasi pandas vs PySpark Local Mode

Pada dataset 3.06 juta baris ini, pandas jauh lebih cepat (agregasi in-memory pandas selesai dalam ~1.00 detik, sedangkan PySpark memerlukan 2.66 detik untuk count() dan 8.44 detik untuk kueri spark agregasi). Tipping point berbalik saat ukuran data melampaui kapasitas RAM fisik mesin (>12 GB RAM). PySpark terbebani overhead internal (JVM initialization, serialization, shuffle partition management) pada data kecil skala MB, namun menjadi solusi mutlak saat data terdistribusi di klaster multi-node.

N.6 Kualitas Data & Pembersihan (Veracity)

Dari total 3.066.766 baris data awal, sebanyak 79.855 baris (2.60%) dibuang karena anomali: jarak <= 0 (45.862 baris / 1.50%), total_amount <= 0 (25.772 baris / 0.84%), durasi > 180 menit (3.048 baris / 0.10%), dan durasi <= 0 (1.121 baris / 0.04%). Pembuangan baris jarak <= 0 berpotensi menimbulkan bias sistemik jika sebagian baris tersebut merupakan tarif flat murni (misalnya rute bandara) yang sinyal GPS-nya gagal mencatat jarak, sehingga menurunkan perkiraan total trip tanpa meteran.

3. Studi Kasus: Penjadwalan Armada Taksi (O)

1. Rekomendasi Jam Sibuk: Manajemen operasional wajib menempatkan konsentrasi armada tertinggi pada pukul 17.00–18.00 karena volume permintaan mencapai puncaknya di angka 210.858 perjalanan per jam, dengan batasan bahwa data ini berasal dari bulan musim dingin (Januari) sehingga tren fluktuasi cuaca perlu diperhitungkan.

2. Rekomendasi Re-alokasi Shift: Pengemudi dari shift malam/dini hari pukul 03.00–04.00 (yang hanya mencatat volume terendah 16.618 perjalanan) sebaiknya dialihkan ke shift sore hari untuk memaksimalkan utilitas armada, dengan batasan bahwa cakupan layanan standar minimum malam hari tetap dipertahankan.

3. Rekomendasi Hari Kerja vs Akhir Pekan: Konsentrasi armada pada jam sibuk pagi (07.00–09.00) wajib diprioritaskan pada hari kerja (Senin–Jumat) karena permintaan pada hari kerja mencapai 114.064 perjalanan di jam 08.00 dibanding akhir pekan yang cenderung landai, dengan batasan bahwa durasi kemacetan harian belum dimasukkan dalam perhitungan rasio per turn-around time.

4. Tugas Praktikum (Q.1 s.d. Q.4)

Q.1 & Q.2 Perbandingan Dua Bulan (Januari vs Juli 2023)

Analisis komparatif dilakukan dengan memuat dataset bulan Juli 2023 (yellow_tripdata_2023-07.parquet) dan menerapkan kriteria penyaringan anomali yang identik. Berikut adalah tabel perbandingan volume perjalanan per jam:

Jam (0-23)	Januari 2023 (Musim Dingin)	Juli 2023 (Musim Panas)	Selisih (Jul - Jan)
0	82,289	85,987	+3,698
1	57,766	59,980	+2,214
2	40,445	40,812	+367
3	26,146	26,000	-146
4	16,618	16,637	+19
5	16,940	16,913	-27
6	42,300	38,422	-3,878
7	84,569	69,358	-15,211
8	114,064	103,647	-10,417
9	127,988	118,788	-9,200
10	140,230	130,529	-9,701
11	150,539	143,780	-6,759
12	165,714	157,384	-8,330
13	174,183	161,822	-12,361
14	186,741	169,475	-17,266
15	191,392	172,601	-18,791
16	190,689	173,983	-16,706
17	204,399	186,220	-18,179
18	210,858	195,988	-14,870
19	188,347	175,235	-13,112
20	161,789	151,925	-9,864
21	157,770	156,152	-1,618

22	143,800	146,257	+2,457
23	111,335	119,762	+8,427

Q.3 Laporan Kinerja Eksekusi

Berdasarkan log pengukuran eksekusi (pengukuran_kinerja.csv), langkah 'tulis CSV' merupakan langkah yang paling mahal dengan durasi eksekusi mencapai 39.19 detik, disusul oleh 'spark agregasi' (8.44 detik) dan 'spark count' (2.66 detik). Untuk mempercepat langkah termahal ini, diusulkan perubahan konkret dengan menghindari penulisan data sementara dalam format CSV teks terkompresi, melainkan menggunakan format biner kolumnar seperti PyArrow IPC Stream atau Parquet terbagi (chunked Parquet), yang sanggup memangkas waktu penulisan I/O disk hingga di atas 80%.

Q.4 Refleksi Teori 5V Big Data

Evaluasi skala dataset berdasarkan dimensi 5V Big Data:

Dimensi 5V	Kategori Skala	Bukti Angka & Alasan Teknis
Volume	Big	Dataset berisi 3.066.766 baris dengan memory footprint mentah 565.6 MB, cukup besar untuk menguji batas efisiensi RAM pada mesin komputasi tunggal.
Veracity	Big	Ditemukan 79.855 baris anomali kotor (2.60%), mencakup durasi negatif (-29.2 menit) dan tarif minus (-$751) yang memerlukan validasi logis ketat.
Velocity	Not Big	Data dipublikasikan secara batch bulanan oleh NYC TLC, bukan data streaming real-time berlatensi tinggi (milidetik).
Variety	Not Big	Struktur data berupa tabel terstruktur (tabular relational Parquet) dengan 19 kolom

		terdefinisi rapi, bukan data semi-terstruktur atau unstructured.
Value	Not Big	Pemrosesan data terbatas pada agregasi statistik deskriptif harian dasar (count, mean, median), belum menyentuh pemodelan prediktif machine learning.

5. Pertanyaan Evaluasi (R.1 s.d. R.8)

R.1 Mengapa ukuran data saja tidak cukup untuk menyebut masalah Big Data?

Big Data bukan sekadar tentang ukuran file giga/terabita, melainkan keterbatasan sumber daya infrastruktur dan kompleksitas pemrosesan saat diolah pada satu mesin. Contoh data kecil yang sulit diolah: Data matriks terhubung (dense graph) berukuran 100 MB. Algoritma perataan lintasan terpendek (All-Pairs Shortest Path / Floyd-Warshall O(V^3)) pada graf tersebut memerlukan alokasi memori berulang dan komputasi CPU masif yang dapat menyebabkan krisis memori (out-of-memory crash) meskipun file fisiknya kecil.

R.2 Dimensi 5V mana yang paling menonjol pada dataset ini?

Dimensi Volume dan Veracity. Volume dibuktikan dengan angka 3.066.766 baris perjalanan taksi (565.6 MB uncompressed RAM). Veracity dibuktikan secara eksplisit dengan ditemukannya 79.855 baris kotor (2.60%) yang mengandung nilai tak logis seperti jarak 0, durasi minus, dan tarif bernilai negatif.

R.3 Alasan teknis Parquet lebih kecil dan cepat dibaca dibanding CSV untuk analitik.

1. Storage Kolumnar & Column Pruning: Parquet menyimpan data per kolom. Saat kueri analitik hanya membutuhkan 8 kolom dari 19 kolom, reader hanya menyentuh blok data relevan dari disk, memangkas I/O disk dan waktu dekompresi. CSV mewajibkan pemindaian seluruh baris.
2. Kompresi Biner Native: Parquet menerapkan dictionary encoding dan algoritma kompresi biner (Snappy/Gzip) pada data berulang, sedangkan CSV menyimpan nilai sebagai karakter teks ASCII yang memboroskan ruang penyimpanan.

R.4 Strategi mengolah CSV 40 GB pada RAM 10 GB (3 Strategi & Kelemahan).

1. Chunking / Streaming (pd.read_csv dengan chunksize): Memproses file per potongan kecil. Kelemahan: Waktu I/O disk relatif lambat dan tidak dapat menghitung metrik non-asosiatif (seperti median eksak) secara bertahap.
2. Out-of-Core Engine (DuckDB / Polars / Dask): Menggunakan mesin kueri SQL/DataFrame

khusus out-of-core. Kelemahan: Membutuhkan penyesuaian sintaks kode dan manajemen tipe data yang lebih ketat.
3. Konversi Format ke Parquet Bertahap: Mengonversi potongan CSV menjadi file Parquet terpisah lalu dikueri. Kelemahan: Membutuhkan ruang disk sekunder yang cukup besar saat proses konversi berlangsung.

R.5 Lazy Evaluation pada Spark dan pengaruhnya pada pengukuran waktu.

Lazy evaluation berarti Spark tidak langsung mengeksekusi komputasi fisik saat perintah transformasi (filter, withColumn, groupBy) dipanggil, melainkan hanya menyusun rencana eksekusi (Directed Acyclic Graph / DAG). Komputasi beneran baru berjalan saat action (count, show, save) dipanggil. Pengaruhnya pada pengukuran waktu: Mengukur waktu di sel transformasi akan selalu menghasilkan durasi ~0 detik (palsu). Pengukuran waktu yang valid wajib diletakkan pada panggilan fungsi action.

R.6 Efek mengabaikan presisi float64 ke float32 dan batas penggunaannya.

Mengubah float64 ke float32 menghemat memori sebesar 50% per nilai numerik (dari 8 byte ke 4 byte per sel). Pengubahan ini tidak boleh dilakukan pada perhitungan akuntansi billing, pembayaran transaksi, atau sistem pencatatan keuangan presisi tinggi, karena float32 mengalami keterbatasan pembulatan desimal (floating-point rounding error) setelah ~7 digit signifikan.

R.7 Mengapa Shuffle disebut operasi termahal dalam pemrosesan terdistribusi?

Shuffle (ditandai dengan operator Exchange pada Spark explain plan) memindahkan data antar partisi atau antar node fisik melalui jaringan. Operasi ini melibatkan serialisasi data di simpul asal, penulisan data sementara ke disk (disk spill), transfer antarmuka jaringan (network I/O), serta penyortiran ulang data di simpul tujuan.

R.8 Mengapa folder /content Colab tidak boleh dijadikan penyimpanan akhir?

Folder /content bersifat ephemeral (sementara) dan akan terhapus secara otomatis ketika runtime berakhir atau terputus (idle reset). Konsekuensinya pada desain notebook: Notebook harus dirancang reproducible, yakni menyertakan skrip otomatisasi mounting Google Drive (drive.mount), pembuatan folder otomatis (os.makedirs), pengunduhan dataset mandiri, serta penyimpanan seluruh file artefak hasil langsung ke Google Drive.